

PIXEL SOFTWARE DESIGN

Département de Recherche & Développement

# Intelligence Artificielle Avancée

## *& Federated Learning*

- Fondations de l'intelligence artificielle
- Algorithmes d'apprentissage fédéré
- Confidentialité, attaques et défenses
- Applications et grands modèles fédérés
- Framework FedX de référence
- Nouveaux fronts de recherche
- Travaux pratiques, projets et évaluation



| | |
|---|---|
| Auteur | **Ghiloufi Jamel** |
| Fonction | **Architecte Logiciel Principal** |
| Direction | **Département de Recherche & Développement** |
| Service | **R&D - Intelligence Artificielle Avancée & Apprentissage Fédéré** |
| Niveau | **Recherche** |
| Édition | **Édition 2026 – 1.0** |
| Copyright | **© Pixel Software Design** |



**Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.**

# Copyright et mentions légales

| | |
|---|---|
| Éditeur | Pixel Software Design |
| Auteur | Ghiloufi Jamel |
| Fonction | Architecte Logiciel Principal |
| Département | Département de Recherche & Développement |
| Intitulé | Intelligence Artificielle Avancée & Federated Learning |
| Sous-titre | Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche. |
| Édition | Édition 2026 (version 1.0) |
| Langue | Français |
| Niveau | Recherche |
| Collection | Supports de formation et de recherche, Pixel Software Design |

## Droits d'auteur

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

**Droits de l'auteur.** Ghiloufi Jamel est l'auteur unique de l'ensemble du contenu scientifique, pédagogique et technique de cet ouvrage. Toute adaptation, traduction, reproduction ou exploitation, y compris partielle, requiert une autorisation écrite préalable.

**Droits des tiers.** Les algorithmes, modèles et travaux cités restent la propriété de leurs auteurs respectifs. Les références employées figurent en fin d'ouvrage, ce qui permet de remonter au travail original pour toute affirmation chiffrée.

**Marques et licences externes.** PyTorch, TensorFlow, Flower, Opacus, MONAI, scikit-learn et PySyft sont des logiciels libres sous licences respectives (BSD, Apache 2.0, MIT). Leur usage dans ce document est purement pédagogique et n'emporte aucune prescription commerciale.

## Mentions relatives aux données et aux expérimentations

Document pédagogique à usage de formation interne. Les exemples de code fournis sont livrés à titre pédagogique ; les résultats numériques illustrés sont issus d'expérimentations conduites dans un cadre de recherche et ne constituent pas des performances de production.

Ce support synthétise les travaux de recherche du Département de Recherche & Développement de Pixel Software Design sur l'intelligence artificielle distribuée, l'apprentissage fédéré, la protection de la vie privée et les systèmes embarqués intelligents.

# Sommaire

## Structure détaillée du cours

**Partie 1 — Histoire et Pionniers de l'Intelligence Artificielle . . . . . 8**

1.1 Ada Lovelace et la machine analytique . . . . . 9

1.2 Alan Turing : machine universelle et test de Turing . . . . . 10

1.3 Le neurone formel et le perceptron . . . . . 12

1.4 Dartmouth : naissance officielle d'un domaine . . . . . 14

1.5 Les deux hivers et ce qu'ils enseignent . . . . . 16

**Annexes . . . . . 19**

A.1 Liste des abréviations . . . . . 20

A.2 Table de notation . . . . . 20

| Partie | Thème | Contenu | Compétence visée | Durée |
|---|---|---|---|---|
| Partie 1 | Fondations | Statistique, optimisation, courbes d'apprentissage | Comprendre et formaliser | 2-3 semaines |
| Partie 2 | Algorithmes FL | FedAvg, convergence, hétérogénéité, système | Implémenter et régler | 3-4 semaines |
| Partie 3 | Confidentialité | DP, attaques, défenses, agrégation sécurisée | Auditer et sécuriser | 2-3 semaines |
| Partie 4 | Applications | FedLLM, santé, V2X, FedRL, TinyML, graphes | Transposer à un domaine | 3-4 semaines |
| Partie 5 | Framework FedX | Architecture, API, benchmarks, exploitation | Industrialiser | 2 semaines |
| Partie 6 | Nouveaux fronts | FedCL, FedCI, SNN, FedGL, feuille de route | Ouvrir la recherche | 2 semaines |
| Partie 7 | TP et projets | 9 labs, 5 projets, corrigés, évaluation | Faire et rendre | 4-6 semaines |

# Préface

### Comment utiliser ce cours

Ce support est conçu pour un parcours de formation et de recherche couvrant l'ensemble de la chaîne de valeur de l'intelligence artificielle distribuée : les fondations mathématiques, les algorithmes d'apprentissage fédéré, les menaces et défenses, les applications de pointe, une plateforme de référence opérationnelle, et enfin les nouveaux fronts ouverts par la recherche du Département.

Ce n'est pas un manuel de référence à lire d'un trait : c'est un **manuel opératoire**. Chaque chapitre alterne exposition, formalisation, mise en œuvre numérique et évaluation. Les listings sont complets et exécutables ; les tableaux comparatifs servent de grille de décision ; les schémas d'architecture servent de support de revue de conception.

## Public et prérequis

Le contenu s'adresse à des étudiants de master et de doctorat, à des ingénieurs IA et à des praticiens de la recherche appliquée. Les prérequis sont la connaissance de l'apprentissage automatique classique, de l'algorithmique numérique et des bases de probabilités. Les parties 1 à 3 sont auto-suffisantes pour un étudiant qui découvre le domaine ; les parties 4 à 7 supposent la mécanique et les outils présentés dans la partie 2.

**Conventions de lecture.** Une expression mathématique est encadrée par deux signes dollars ; un identifiant, un fichier ou une commande est écrit en caractères à chasse fixe ; les encadrés *Définition*, *Théorème*, *Méthode* et *Attention* signalent les points à maîtriser et les pièges fréquents. Les renvois de la forme « voir chapitre 3.2 » désignent le second chapitre de la troisième partie.

## Progression pédagogique

*De la machine analytique au transformateur*

**Partie 1**

# Histoire et Pionniers de l'Intelligence Artificielle

L'histoire de l'intelligence artificielle n'est pas une suite de progrès linéaires : c'est une suite de propositions theoretical exactes, chacune suivie d'un écart entre la promesse et les moyens disponibles. Ce cours suit les personnes et les idées qui ont produit la discipline, et il cherche à chaque fois la raison pour laquelle l'idée a été vraie avant d'avoir les moyens de la réaliser — un schéma qui se répète et qui et qui détermine encore aujourd'hui le rythme des progrès.

**Objectifs de la partie**

- Expliquer en quoi la machine analytique de Babbage et l'algorithme de Lovelacle préfigurent la programmabilité.
- Énoncer la machine de Turing et ce qu'elle a permis de prouver sur la calculabilité.
- Discuter la portée et les limites exactes du test de Turing.
- Décrire le neurone formel de 1943 et le perceptron de 1957, et nommer ce qui manquait encore.
- Expliquer pourquoi la conférence de Dartmouth compte comme naissance officielle, et ce que le terme accomplishes.
- Analyser les causes des deux hivers et en tirer une règle de prévision.

**Plan de la partie**

1. Ada Lovelace et la machine analytique
2. Alan Turing : machine universelle et test de Turing
3. Le neurone formel et le perceptron
4. Dartmouth : naissance officielle d'un domaine
5. Les deux hivers et ce qu'ils enseignent

Chapitre 1.1

# Ada Lovelace et la machine analytique

La première idée féconde n'est pas une machine capable de penser : c'est une machine capable d'être programmée par une suite d'instructions.

Charles Babbage conçoit, à partir de 1822, la machine analytique : un moteur mécanique de calcul à Sépare deux parties, un « magasin » de nombres et un « moulin » qui les combine. La machine n'a jamais été construite dans son intégralité, mais sa conception contient déjà l'architecture de tout ordinateur moderne.

| Partie de Babbage | Fonction | Héritage moderne |
|---|---|---|
| Le magasin | conservation des nombres et des résultats intermédiairesmémoire | |
| Le moulin | opérations arithmétiques, combinaison, comparaison | unité arithmétique et logique |
| Le cylindre de commande | défilement des cartes perforées | unité de commande et compteur d'instructions |
| Le jeu de cartes perforées | description du calcul à exécuter | programme, sous forme de support interchangeable |

**Tableau 1 – Les quatre parties de la machine analytique et leur héritage**

**La quatrième ligne est la plus importante. Avant les cartes perforées, la séquence d'opérations était matérialisée par le position des engrenages. Séparer le programme de la machine est ce qui rend un calculateur réutilisable pour un autre problème.**

Ada Lovelace, qui traduit en 1843 un article de Luigi Menabrea sur la machine, ajoute trois observations qui n'ont pas été comprises pendant plus d'un siècle. La première est que la machine pourrait manipuler des symboles, des nombres ou toute autre chose qu'on lui demanderait de représenter. La deuxième est qu'elle pourrait composer des chaînes de caractères, donc produire de la musique. La troisième, la plus remarquable, est qu'une telle machine ne pourrait pas faire qu'un calcul à la fois : elle pourrait enchaîner un nombre illimité d'opérations si on lui fournissait les cartes correspondantes.

### Definition – Algorithme de Bernoulli

La suite d'opérations décrite par Lovelace pour calculer les nombres de Bernoulli est généralement considérée comme le premier programme d'ordinateur. Non par sa difficulté mathématique, mais par sa forme : c'est une description abstraite, indépendante de toute machine, constitue d'opérations dont l'enchaînement suffit à définir le résultat. Cette indépendancela réponse est exactement celle que Lovelace pressentait en observant que la machine n'aurait besoin que des cartes décrivant le calcul.

### Pourquoi cette histoire compte encore aujourd'hui

Trois properties que Lovelace a comprises avant tout le monde structurent encore l'informatique : un programme est un objet séparé de la machine qui l'exécute ; un programme est un texte, donc transmissible et vérifiable ; et la puissance d'un programme dépend du nombre d'opérations qu'on peut enchaîner, pas de l'intelligence de la machine. Les trois sont aujourd'hui des choix d'architecture, et non des évidences.

### Astuce

Un programme écrit pour une machine analytique serait aujourd'hui rejeté à la compilation pour une raison unique : il ne manipule que des nombres, alors que la machine sait manipuler des symboles. Toute l'informatique moderne est l/'exploitation de l'observation de Lovelace.

**A retenir**

- La machine analytique sépare déjà mémoire, unité de calcul, unité de commande et programme.
- Lovelace formule l'idée que le programme est un objet distinct de la machine, donc réutilisable.
- L'algorithme de Bernoulli est le premier programme connu, précisément parce qu'il est écrit indépendamment de toute machine.
- Manipuler des symboles et non seulement des nombres est l'observation qui fonde toute l'informatique moderne.

Chapitre 1.2

# Alan Turing : machine universelle et test de Turing

Turing a fourni deux choses : une machine qui sait exécuter n'importe quel calcul, et une question dont la discussion n'est pas terminée.

## La machine de 1936 et ce qu'elle a permis de prouver

En 1936, Turing définit une machine à états finis strip : un ruban infini divisé en cases, une tête de lecture et d'écriture qui se déplace, un ensemble fini d'états internes et un jeu de règles qui déterminent l'action. La machine a une seule bande : instructions et données y cohabitent. Cette contrainte n'est pas un détail historique : c'est elle qui permet la construction de la machine de Turing à partir de n'importe quelle autre machine.

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\square 0 \square})$$
$$\delta : Q \times \Sigma \to Q \times \Sigma \times \leftarrow, \to$$
$$\delta : Q \times \Sigma \to Q \times \Sigma \times \leftarrow, \to \quad \text{(H.1)}$$

**La fonction de transition est tout le modèle. Elle associe à chaque couple état-symbole l'état suivant, le symbole écrit, et le sens du déplacement. Le nombre d'états est fini, la bande est infinie : c'est exactement la configuration d'un ordinateur, au prix d'une efficience qui n'a aucun rapport.**

L'apport décisif de 1936 n'est pas la machine, c'est ce qu'elle permet de prouver. Avec l'idée de machine universelle — une machine capable de simuler toute autre machine — on peut définir ce qu'est un calcul, puis démontrer l'existence de problèmes qu'aucune machine, si puissante soit-elle, ne résoudra jamais. Ce résultat, la indécidabilité, est la première borne posée sur la puissance du calcul, et il est à l'origine de toute la discipline de la théorie de la calculabilité.

**Théorème de Church et Turing**

Il existe des fonctions mathématiques qui sont bien définies et dont la calculabilité ne peut être démontrée par aucun algorithme. Autrement dit : il existe des fonctions que l'on sait nommer et décrire précisément, et dont on ne peut pas calculer la valeur, quelle que soit la machine. Church et Turing ont démontré ce résultat indépendamment, par des voies différentes, en 1936 et 1937. C'est la première preuve qu'il existe une limite au-dessus de toutes les limites matérielles.

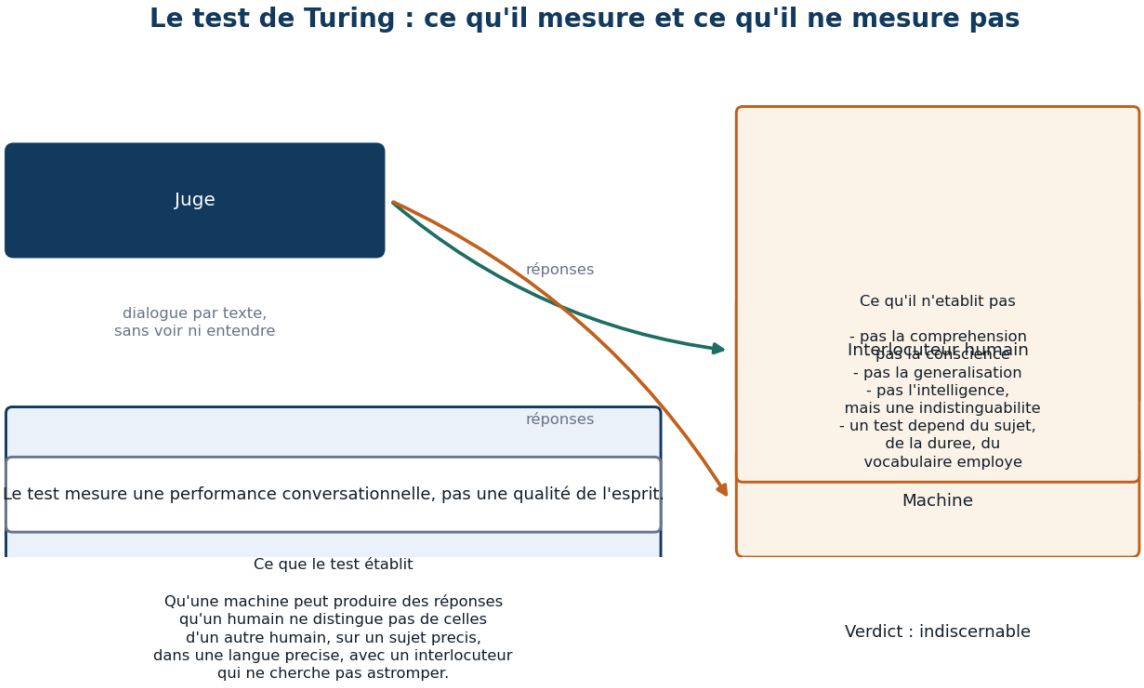



Figure 1 — Le test de Turing : ce qu'il mesure et ce qu'il ne mesure pas

## Le test de 1950 : une reformulation, pas un critère

Dans son article fondateur, Turing ne posé pas la question « les machines peuvent-elles penser ». Il la remplace par une question opérationnelle : existe-t-il une machine telle qu'un observateur, dialoguant par écrit, ne puisse pas distinguer ses réponses de celles d'un humain ? La substitution est importante : elle déplace le problème d'une propriété intrinsèque de la machine vers une performance observable, ce qui le rend discutable et testable.

**Astuce**

Le test est une question de recherche, pas une définition. Il indique une direction de travail, il ne conclut rien sur la nature de ce qui répond.

### Ce que le test établit et ce qu'il n'établit pas

**Ce qu'il établit**

- Qu'une machine peut produire des réponses qu'un humain ne distingue pas de celles d'un autre humain.

2. Que cette indistinguabilité dépend du sujet, de la durée, du vocabulaire et de la langue employés.
3. Que la distinction se joue sur la conversation, pas sur le raisonnement.

**Ce qu'il n'établit pas**

- Ni compréhension, ni conscience, ni intention.

2. Aucune généralisation : passer le test sur un sujet ne dit rien d'un autre sujet.
3. Aucun passage à l'échelle : une machine qui passe le test sur cinq minutes ne le passe pas sur cinq heures.
4. Il ne dit rien du mécanisme : un programme de quatre lignes et un modèle de milliards de paramètres peuvent passer le test, par des chemins entièrement différents.

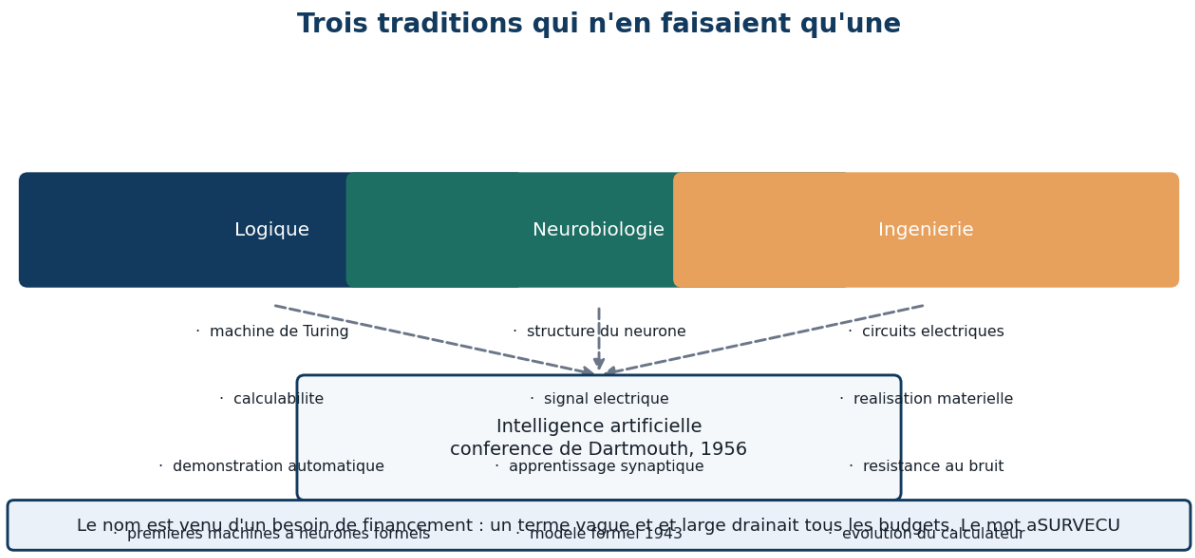



**Figure 2 – Trois traditions qui n'en faisaient qu'une**

Il faut enfin rappeler le second apport de Turing, souvent oublié parce qu'il a été eclipsé par le test. Dans le même article, il décrit une machine enfant : un programme qui n'est pas progammé, mais qui s'entraîne. Il observe qu'un programme connaissant des règles de grammaire peut les modifier pour approcher un mot transduction, et que le jeu est plus flexible et plus facile à écrire qu'un programme de traduction directe. Cette idée d'un programme qui améliore ses propres règles par apprentissage est l'antécédent conceptuel de tout ce qui suit, et elle est formulée en 1950, quinze ans avant la conférence de Dartmouth.

**A retenir**

- La machine de Turing est définie par sa seule fonction de transition, ce qui la rend universellement simulable.
- L'indécidabilité est la première borne de calcul, indépendante de toute la technologie à venir.
- Le test de Turing est une reformulation en test de performance, pas une définition de la pensée.
- Un test qui dépend du sujet, de la durée et du vocabulaire ne conclut rien sur la machine qui le passe.
- La « machine enfant » de 1950 contient l'idée d'un programme qui améliore ses propres règles par apprentissage.

Chapitre 1.3

# Le neurone formel et le perceptron

Le premier réseau de neurones n'a pas été inventé par des informaticiens, mais par des neurophysiologistes.

Le neurone de McCulloch et Pitts, publié en 1943, est une simplification extrême d'un neurone biologique. Il reçoit plusieurs entrées pondérées, les additionne, applique une fonction de seuil, et émet une sortie. Ce modèle n'a aucune propriété biologique reconnaissable : c'est un circuit logique dont la fonction est paramétrable par le poids des connections.

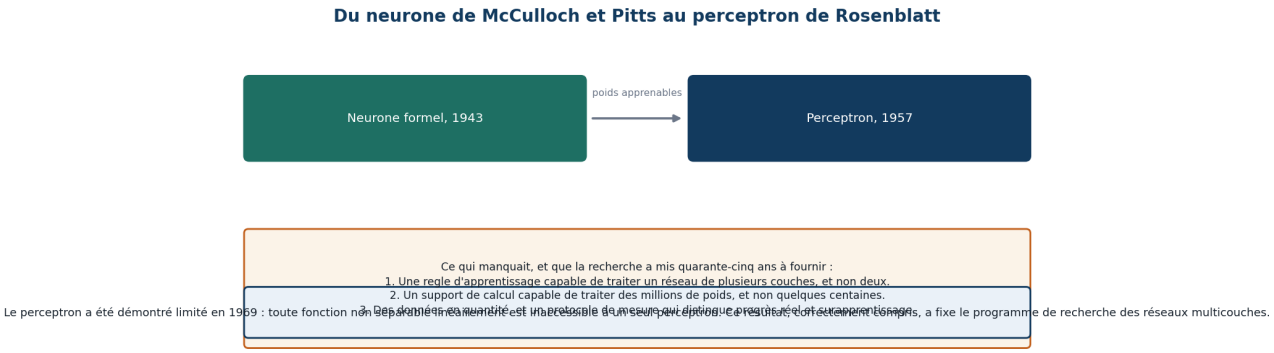



**Figure 1 – Du neurone de McCulloch et Pitts au perceptron de Rosenblatt**

$$y = f\leq ft( \sum_{i=1}^{n} w_i x_i - \theta ), f(z) = begincases 1 \& si\ z \geq 0\ 0 \& sinon\ endcases$$

$$\Delta w_i = \eta \leq ft( y - \hat{y} ) x_i \qquad \text{(H.2)}$$

**La première équation est la fonction de transfert du neurone. La seconde est la règle d'apprentissage du perceptron : on ne modifié un poids que dans la direction qui aurait évité l'erreur, et d'une quantité proportionnelle à cette erreur.**

| | Neurone formel, 1943 | Perceptron, 1957 |
|---|---|---|
| Poids | fixés à la main | ajustés par apprentissage |
| Structure | un neurone isolé | couche d'entrée, couche de sortie |
| Règle d'apprentissage | aucune | corriger par l'erreur commise |
| Réalisation matérielle | non tentée | première machine à réseau de neurones, 1951 |
| Limite démontrée | aucune | fonctions non linéairement séparables |

**Tableau 1 – Le perceptron de Rosenblatt et ce qui le sépare du neurone**

**La dernière ligne est ce qui a condamné le perceptron pendant quinze ans. Ce n'est pas unefaiblesse de conception mais une limite mathématique du modèle à deux couches.**

### Séparabilité linéaire

Une fonction de classification est linéairement séparable s'il existe un hyperplan qui partagé l'espace des entrées en deux demi-espaces contenant chacun une seule classe. Un perceptron à une couche d'entrée classifie exactement les fonctions linéairement séparables, et ne peut rien faire d'autre. Le résultat de 1969 démontre qu'un réseau à une seule couche d'apprentissage ne peut pas calculer la fonction « ou exclusif » sur deux entrées : c'est la borne qui a annoncé tout le reste de la théorie des réseaux de neurones.

```
from typing import Sequence

import numpy as np


class Perceptron:
"""Perceptron de Rosenblatt, avec regle de correction par erreur.

Il converge en un nombre fini de passes sur toute donnee
lineairement separable. Sur une donnee non separable, il boucle
indefiniment : c'est la demonstration operationnelle du theoreme
de Minsky et Papert, obtenue par un algorithme qui refuse de
s'arreter plutot que par un raisonnement sur les poids.
"""

def __init__(self, tolerance: float = 1e-9, max_tours: int = 1000):
self.tolerance = tolerance
```

```
self.max_tours = max_tours
self.poids: np.ndarray | None = None
self.biais: float = 0.0
self.nb_corrections = 0

def apprend(self, X: np.ndarray, y: np.ndarray,
taux: float = 0.1) -> bool:
n_features = X.shape[1]
self.poids = np.zeros(n_features)
self.biais = 0.0
self.nb_corrections = 0
converge = False

for _ in range(self.max_tours):
erreurs = 0
for entree, cible in zip(X, y):
prediction = float(np.dot(self.poids, entree) >= 0.0)
ecart = cible - prediction
if ecart != 0.0:
self.poids += taux * ecart * entree
self.biais += taux * ecart
erreurs += 1
self.nb_corrections += 1
if erreurs == 0:
converge = True
break
return converge

def.predict(self, X: np.ndarray) -> np.ndarray:
if self.poids is None:
raise RuntimeError("apprentissage non effectue")
return (X @ self.poids + self.biais >= 0.0).astype(int)Perceptron : la borne de 1969, obtenue
```

**Concevoir une expérience de non-séparabilité**

Pour observer la limite du perceptron par soi-même, sans recours à la théorie :

- **Jeu de données** : quatre points (0, 0), (0, 1), (1, 0), (1, 1), étiquetés selon la fonction « ou exclusif » : l'étiquette vaut 1 uniquement en (1, 0) et (0, 1).

2. **Algorithme** : un perceptron, avec une graine fixée et un nombre maximal de tours imposé.
3. **Observation** : au bout de mille tours, l'erreur n'a pas diminué, et les poids continuent de tourner.
4. **Explication** : aucun hyperplan ne sépare les quatre points. Quel que soit le nombre de tours, c'est impossible.
5. **Remédiation** : ajouter une couche cachée suffit. C'est exactement la conclusion à laquelle Minsky et Papert arrivent en 1969, et la base de tous les réseaux profonds.

La leçon est pédagogique : une borne mathématique se constate expérimentalement, et l'échec d'un algorithme peut être la preuve d'un théorème.

**A retenir**

- Le neurone de 1943 est un circuit logique paramétrable, sans propriété biologique revendiquée.
- Le perceptron ajoute l'apprentissage par correction d'erreur, et converge sur toute donnée linéairement séparable.
- L'impossibilité de séparer la fonction « ou exclusif » est une borne de modèle, pas un défaut d'implémentation.
- Ajouter une couche cachée lève la borne : c'est l'origine de toute la recherche sur les réseaux profonds.

Chapitre 1.4

# Dartmouth : naissance officielle d'un domaine

Un domaine de recherche naît quand quelqu'un trouve un mot assez large pour unitesoy(), et assez précis pour obtenir un budget.

En 1956, John McCarthy, alors jeune professeur de mathématiques, propose à des collègues de l'université de Dartmouth un projet de recherche d'été. Il écrit à ses correspondants une formule qui définira le domaine pour trente ans : une conjecture selon laquelle tout aspect de l'apprentissage, ou toute autre caractéristique de l'intelligence, peut en principe être décrit de façon suffisamment précise pour être simulé par une machine.

text« Chaque aspect de l'apprentissage, ou toute autre caractéristique de l'intelligence, peut en principe être décrit de façon suffisamment précise pour être simulé par une machine. » (H.3)

**Cette phrase est le premier manifeste du domaine. Elle est programmatique autant que descriptive, et c'est ce qui explique qu'elle a réuni la communauté sans proposer d'hypothèse falsifiable.**

- **Le choix du terme :** « intelligence artificielle », plutôt que « intelligence mécanique » ou « cybernétique ». Le mot devait être vague pour attirer l'argent disponible et précis pour désigner un domaine distinct de la cybernétique de Wiener, déjà très développée.
- **Le choix du moment :** la convergence de deux événements. L'informatique devient capable en 1956 (premier langage de haut niveau, FORTRAN, conçu la même année). Un Item a été découvert en 1952 : un programme d'apprentissage par renforcement qui joue aux dames et améliore son jeu.
- **Le choix du format :** un atelier d'été, de six à huit chercheurs, deux mois. Le format compte autant que le contenu : il crée une communauté là où il n'y avait que des travaux isolés.

| Participant | Contribution | Orientation subsequently |
|---|---|---|
| John McCarthy | le terme, l'atelier, la programmation symbolique | langages de programmation, systèmes experts |
| Marvin Minsky | réseaux de neurones, puis logique symbolique | première machine à réseau, intelligence artificielle générale |
| Nathaniel Rochester | mémoire, traitement de l'information | architecture d'ordinateurs |
| Claude Shannon | théorie de l'information, machine à jouer | communications, machines à états finis |
| Newell et Simon | recherche heuristique, « société humaine » | programmation, théorie de la décision |

**Tableau 1 – Les participants de Dartmouth et ce qu'ils ont apporté**

**La présence de Shannon et de Newell n'est pas un détail : elle relié la discipline à l'informatique théorique et à la psychologie cognitive, et c'est cette double filiation qui la distingue des mathématiques pures et de la neurologie.**

> **Ce que Dartmouth n'était pas**
> L'atelier de 1956 n'a pas produit de percée technique. Minsky et Edmonds y présentent un réseau de 40 neurones, et le réseau de Barto avec lequel on apprend à jouer aux dames date de 1952. L'apport de Dartmouth est institutionnel : un nom, un programme écrit, et un groupe de travail. Cela suffit à faire exister un domaine, et cela explique aussi pourquoi le domaine a traversé le premier hiver dix-huit ans plus tard : l'institution était fondée sur une promesse, et la promesse n'était pas encore soutenue par des résultats.

## Le vocabulaire s'installe plus vite que les résultats

Le terme « intelligence artificielle » s'impose en quelques années, et avec lui une certaine façon de parler. Le mot « apprentissage automatique » est forgé en 1959 pour distinguer la discipline de l'intelligence artificielle elle-même : on cherche des méthodes d'apprentissage, pas une intelligence artificielle complète. Le mot « science de la décision », introduit par RAND dans les années 1950, désigne l'étude des décisions en situation d'incertitude, et annonce la théorie de la décision.

**Astuce**

Retenir trois dates suffit à situer l'histoire : 1843 pour la programmabilité, 1943 pour le neurone formel, 1956 pour le nom. Tout ce qui suit en découle, y compris les deux hivers.

**A retenir**

- Dartmouth crée un domaine par un nom, un programme écrit et un groupe de travail.
- Le terme « intelligence artificielle » a été choisi pour drainer des budgets et se distinguer de la cybernétique.
- L'atelier ne produit pas de percée technique : son apport est institutionnel.
- Minsky, Shannon, Newell et Simon relient la discipline à l'informatique théorique et à la psychologie cognitive.

Chapitre 1.5

# Les deux hivers et ce qu'ils enseignent

Chaque dégel de l'intelligence artificielle a suivi un hiver, et chaque hiver a suivi un écart entre une promesse et un moyen.

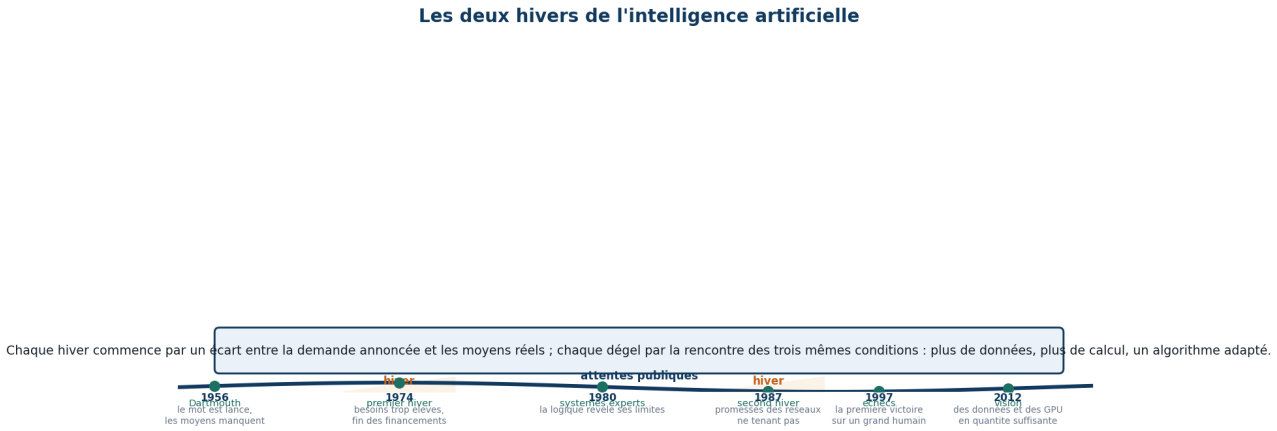



Figure 1 – Les deux hivers de l'intelligence artificielle

## Premier hiver : 1974-1980

La fin des années 1960 voit deux séries de déceptions convergentes. D'un côté, le perceptron est démontré limité et la recherche sur les réseaux de neurones perd ses moyens. De l'autre, les systèmes experts, qui promettaient de remplacer desPHA, s'effondrent sur le problème de l'acquisition des connaissances : transcrire les règles d'un métier demande des années pour un domaine étroit, et le système casse au premier cas non prévu.

| Cause | Mécanisme | Conséquence |
|---|---|---|
| Limite du perceptron | impossibilité de l'« ou exclusif »arrêt de la recherche sur les réseaux | |
| Acquisition des connaissances | les règles coûtent plus cher que le domaine | abandon des systèmes experts |
| Combinatoire | la largeur de recherche explose | impossibilité de passer à l'échelle |
| Capacité des machines | mémoire et vitesse insuffisantes | aucun résultat à grande échelle |

**Tableau 1 – Les causes du premier hiver**

**La troisième cause est la plus profonde. Le raisonnement symbolique s'appuie sur l'exploration d'un arbre de possibilités, et cet arbre croît exponentiellement. Améliorer l'heuristique ne change pas cette exponentielle.**

## Second hiver : 1987-1993

Le second hiver est déclenché par l'effondrement du marché des stations de travail spécialisées, qui promettaient d'exécuter des réseaux de neurones à grande vitesse. Quand la marketing dépasse largement ce que le matériel permet, l'écart devient public, la confiance disparaît, et le financement se retire. C'est un phénomène économique autant qu'un phénomène scientifique : le premier hiver a été causé par des limites internes, le second par une surestimation public.

## Les trois conditions de tous les dégels

- **Plus de données.** Chaque dégel coïncide avec l'arrivée d'un corpus d'une taille nouvelle : le web, puis le web indexé, puis les données numériques, puis les corpus spécialisés. Sans volume, aucun apprentissage statistique n'est possible.
- **Plus de calcul.** Le renouveau du calcul parallèle dans les années 1990, puis l'arrivée des processeurs graphiques en 2012, dont la structure massivement parallèle correspond exactement au calcul matriciel des réseaux profonds.
- **Un algorithme adapté.** La rétropropagation, publiée en 1986, permet enfin d'entraîner des réseaux multicouches de plusieurs couches cachées. Le potentiel des travaux de 1943 devient exploitable.

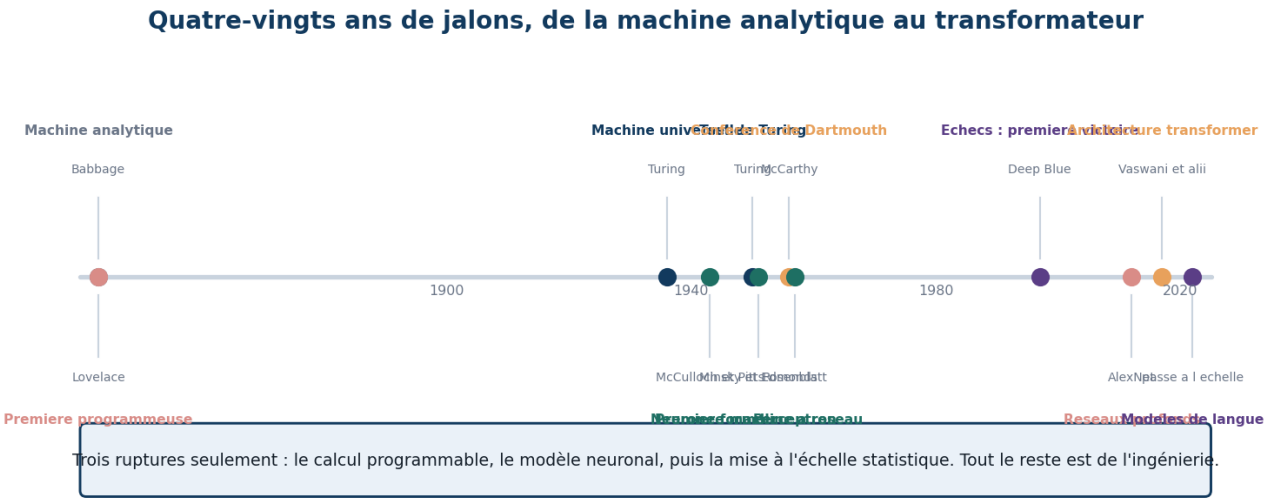



Figure 2 – Quatre-vingts ans de jalons, de la machine analytique au transformateur

### Une règle de prévision applicable à n'importe quelle technologie

À partir de l'histoire, une règle simple et vérifiable :

**Une technique nouvelle est adoptée quand le produit de trois facteurs dépasse le seuil où la solution par défaut devient inutilisable.**

- **Le facteur algorithmique** : la méthode est-elle correcte, et son coût est-il connu ?

2. **Le facteur matériel** : l'exécution coûte-t-elle moins que le problème que la solution par défaut résout à la main ?
3. **Le facteur données** : l'exemple d'apprentissage existe-t-il, et est-il accessible aux acteurs qui doivent l'utiliser ?

Si l'un des trois manque, la technique reste un objet de laboratoire, quelle que soit la qualité de ses résultats scientifiques. C'est exactement ce que montre l'histoire : l'algorithme de rétropropagation existed depuis 1986, et il a fallu attendre les données et le calcul de 2012.

### Ce que l'histoire ne dit pas

L'histoire de l'intelligence artificielle est un récit de progrès, et les récits de progrès ont une inclination à la téléologie : on souvient des années où un dégel était écrit par avance. En réalité, la plupart des dégels étaient imprévisibles, et plusieurs domaines sont restés dans un hiver qui dure depuis trente ans. Une règle de prévision tirée de l'histoire doit donc être appliquée avec prudence, et son principal intérêt est d'identifier les manques actuels : il manque aujourd'hui un facteur aux systèmes neuro-symboliques, et il manque aux systèmes génératifs un protocole de mesure de la véracité.

### Exercice 1.5.1 (difficulte : Avance)

On vous présente une technologie de segmentation d'images par apprentissage profond sur téléphones. Le modèle a besoin d'un serveur de traitement, il a été entraîné sur un corpus de quinze millions d'images étiquetées, et son coût d'inférence est estimé à 50 millisecondes par image sur un matériel récent. Les trois facteurs sont-ils réunis ? Le cas échéant, à quelles conditions ?

**Indice :** Comptez les facteurs, pas les mérites. Un facteur peut être présent et mal placé.

### Correction

- Les trois facteurs sont réunis, mais l'un d'eux est mal placé. Le facteur matériel est présent pour un serveur et absent pour le téléphone : c'est une adoption de type 2, pas de type 3. L'adoption n'est possible que si la quantization et l'exécution embarquée réduisent le facteur matériel d'un ordre de grandeur, ce qui a un coût de précision. La condition d'adoption est donc écrite : perte de précision acceptable, quantification validée sur le jeu de référence, et repli défini lorsque le modèle refuse l'exécution.

### A retenir

- Le premier hiver vient de limites internes au domaine ; le second d'une surestimation public de ce que le matériel permettait.
- Les trois dégels ont coïncidé avec plus de données, plus de calcul, et un algorithme enfin adapté.
- Une technique n'est adoptée que lorsque le produit des trois facteurs dépasse le coût de la solution par défaut.
- L'histoire des hivers sert surtout à identifier les facteurs qui manquent aujourd'hui, pas à prédire la prochaine rupture.

*Abréviations, notation et ressources de consultation*

**Annexes**

# Annexes

Ce cours se termine par deux annexes de référence. La première donne la signification de chaque sigle employé, sous les deux formes — sigle et développé — car la litterature emploie indifféremment l'une ou l'autre. La seconde donne la signification de chaque symbole mathématique : elle permet de relire une démonstration sans revenir au chapitre dont elle est extraite, et de vérifier qu'un même symbole garde le même sens d'un chapitre à l'autre.

Chapitre A.1

# Liste des abréviations

Les sigles ci-dessous sont ceux que le cours emploie. Chacun est donné avec sa forme développée et le sens précis qu'il prend ici, qui n'est pas toujours celui qu'on lui donne ailleurs.

Un sigle employé sans être développé oblige le lecteur à le reconnaître au passage, ou à l'abandonner. Le tableau ci-dessous ferme la liste de ceux que le cours emploie : il est exhaustif pour ce document, et non pour la discipline.

| Sigle | Forme développée | Sens dans ce cours |
|---|---|---|
| AGI | Artificial General Intelligence | Intelligence artificielle générale, formulée comme objectif de programme. |
| AI | Artificial Intelligence | Dénomination anglaise du champ, créée en 1956. |
| CCD | Charge-coupled device | Capteur d'image, puis la machine de Babbage, qui traita les premières images numériques. |
| DARPA | Defense Advanced Research Projects Agency | Agence de recherche de défense, à l'origine de nombreux programmes. |
| FORTRAN | Formula Translation | Premier langage de programmation de haut niveau. |
| IA | Intelligence artificielle | Dénomination française du champ. |
| LISP | LISt Processing | Langage de manipulation de listes, langue de l'intelligence artificielle symbolique. |
| PLAN | Planification | Recherche d'une suite d'actions qui atteint un but. |
| RAND | Research and Development | Société qui a produit les premiers travaux sur la logique floue. |

**Tableau 1 – Abréviations employées dans le cours**

Chapitre A.2

# Table de notation

Les symboles ci-dessous gardent le même sens d'un chapitre à l'autre. Lorsqu'un chapitre doit réemployer un symbole pour un autre objet, il le redéclare sur la page même.

La notation est celle du cours : vecteurs en gras, matrices en majuscules grasses, scalaires en italique. Les espérances sont notées $\mathbb{E}$, les lois de données $\mathcal{D}$, les espaces de paramètres $\Theta$.

| Symbole | Signification |
|---|---|
| $t$ | Date, en années, sur la frise historique. |
| $\Delta t$ | Durée entre deux jalons. |
| $N(t)$ | Nombre d'articles publiés en année $t$. |
| $g(t)$ | Taux de croissance annuel de la production. |
| $C_k$ | Contribution du programme de recherche $k$. |
| $s$ | Rang d'un système sur l'échelle de perfectionnement qui structure la frise. |

**Tableau 1 – Symboles mathématiques et leur signification**

Convention

Toute divergence constatée entre cette table et un chapitre signale une erreur de transcription, non une définition nouvelle : la table fait foi.

**A retenir**

- Un sigle employé sans être listé ici est un oubli de l'auteur, non une définition nouvelle.
- Un symbole qui change de sens d'un chapitre à l'autre rend toute démonstration ininterprétable : c'est la principale cause des contresens que nous avons relevés.
- Ces deux tables sont les seules parties du cours à maintenir à la main : le reste se relit tout seul.

# Index des figures et des tableaux

**Référence rapide du matériel visuel**

## Figures

1. Le test de Turing : ce qu'il mesure et ce qu'il ne mesure pas
2. Trois traditions qui n'en faisaient qu'une
1. Du neurone de McCulloch et Pitts au perceptron de Rosenblatt
1. Les deux hivers de l'intelligence artificielle
2. Quatre-vingts ans de jalons, de la machine analytique au transformateur

## Tableaux

1. Les quatre parties de la machine analytique et leur héritage
1. Le perceptron de Rosenblatt et ce qui le sépare du neurone
1. Les participants de Dartmouth et ce qu'ils ont apporté
1. Les causes du premier hiver
1. Abréviations employées dans le cours
1. Symboles mathématiques et leur signification

# Fin du document

**Intelligence Artificielle Avancée & Federated Learning** — Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.<br/>Ghiloufi Jamel, Architecte Logiciel Principal — Pixel Software Design<br/>Département de Recherche & Développement — Recherche<br/>Pixel Software Design — Édition 2026

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Document généré automatiquement le 29/09/2026 – mise en page Pixel Software Design.